Task 3: Dynamic Signature Verification

Name: SUPRAJEET VIJAY PATIL

Matriculation Number: 100004941

Date of Submission: 17.12.2025

Introduction

Handwritten signatures are one of the most widely used biometric traits for personal authentication due to their long-standing acceptance in legal, financial, and administrative procedures. Unlike static (offline) signatures, which are captured as images, **dynamic (online) signatures** record the writing process itself, including pen position, timing, and pen-up/pen-down information. This additional temporal information makes dynamic signature verification more robust against skilled forgeries and variations in writing style.

Dynamic signature verification can be formulated as a **pattern matching problem**, where each signature is represented as a sequence derived from the writing trajectory. Because no two genuine signatures are the same, the verification system must be able to tolerate natural variations while still discriminating between genuine and forged signatures. For this reason, sequence comparison techniques have been widely studied in literature.

In this project, a **graphical and string-based representation** of dynamic signatures is adopted, following the methodology introduced in Chapter 6 of the lecture notes. Each signature is first decomposed into individual strokes using pen-up and pen-down information. From each stroke, symbolic descriptors are extracted based on local extrema in horizontal and vertical directions. This process transforms a continuous handwriting signal into a sequence of symbolic strings that capture the structural characteristics of the signature.

To compare two signatures, an **adapted Levenstein (Minimum Edit Distance) algorithm** is employed. Unlike the standard edit distance used for text strings, the adapted version operates at the stroke level and incorporates the internal structure of each stroke. Insertions and deletions are penalized according to stroke length, while substitutions are evaluated using the edit distance between symbolic stroke representations. The resulting distance is then normalized, allowing meaningful comparison between signatures of different lengths.

The implemented system performs verification experiments on the provided dynamic signature dataset, automatically detecting multiple user sets (e.g., U1, U2, U3, etc.). The evaluation includes **genuine-to-genuine comparisons** as well as **genuine-to-forged or impostor comparisons**, producing clear ACCEPT or REJECT decisions based on a fixed threshold. By printing detailed comparison results, the system provides transparency into how signature similarity varies between genuine and forged cases.

Overall, this project demonstrates how dynamic signature verification can be effectively implemented using sequence-based representations and adapted edit distance techniques. The approach is computationally simple, interpretable, and closely aligned with the theoretical framework presented in the lecture notes, making it suitable for both academic study and practical experimentation.

Methodology

This task follows a straightforward pipeline: read the dynamic signature signals, convert them into a symbolic sequence, and then compare two signatures using an adapted Levenstein (minimum edit distance) approach. The goal is to keep the method interpretable while still capturing the main dynamic properties of the writing process.

Procedures Carried Out:

1. **Load and interpret raw online signature data:**
   Each signature sample is stored as a time-ordered list of points. For every point, the dataset provides at least:
   - **x-coordinate** (horizontal position)
   - **y-coordinate** (vertical position)
   - **t** (time index or timestamp)
   - **s** (pen state: pen-down / pen-up

   The file is read line by line and converted into a list of points in the same order as recorded, so the temporal structure of the signature is preserved.

2. **Split the trajectory into strokes (pen-down segments):** A signature is not one continuous curve: the pen is lifted and placed down multiple times. So, the point stream is split into separate **strokes**:

   - while the pen is down → points are appended to the current stroke
   - when a pen-up occurs → the stroke ends and is stored
   - very short fragments are ignored to reduce noise

   After this step, one signature becomes a **sequence of strokes**.

3. **Convert each stroke into a symbolic "string":** Instead of comparing raw coordinates directly, each stroke is turned into a short symbolic description.The idea is to keep something meaningful but simple to explain: the stroke is scanned and we detect **local extrema** in x and y:

   - local minimum in x → `xmin`
   - local maximum in x → `xmax`
   - local minimum in y → `ymin`
   - local maximum in y → `ymax`

   Whenever an extremum is detected (compared to neighboring points), the corresponding symbol is appended.So each stroke becomes a **string of tokens**, and the whole signature becomes a **sequence of token-strings**.

4. **Compare two strokes using standard Levenstein distance:** Two strokes are compared by computing the classic **Levenstein (edit) distance** between their token strings:
   "How many insertions, deletions, and substitutions are needed to change stroke A's symbol string into stroke B's symbol string?" This part handles natural variations well (small differences in the extrema sequence lead to a small edit distance).

5. **Compare two signatures using the adapted Levenstein distance:** Now we compare at the *signature level*, meaning we compare **two sequences of strokes**. This is where the "adapted" part comes in: the dynamic programming alignment is done over strokes, with these costs:

   - delete a stroke: cost proportional to the length (number of symbols) of that stroke
   - insert a stroke: cost proportional to the length of the inserted stroke
   - replace a stroke: cost = Levenstein distance between the two stroke-strings (from Step 4)

   This produces one final distance value that reflects both:

   - differences in the number of strokes, and
   - differences inside matching strokes.

6. **Normalize the distance to make scores comparable:** Long signatures naturally produce larger raw edit distances.To avoid that bias, the distance is normalized by the total number of symbols in both signatures (so the score becomes a fair "relative mismatch" rather than a length-dependent number).

7. **Decision rule and reporting:**
   For verification, a simple threshold rule is used:
   - **If the normalized distance is ≤ threshold → ACCEPT**
   - **Otherwise → REJECT**

   To make the results easier to interpret, the program prints comparisons such as:
   - **Genuine vs Genuine** (same user set)
   - **Genuine vs Forged** (if forged samples exist in the dataset structure)
   - If no explicit forgeries exist, it performs **Genuine vs Impostor** comparisons by taking a sample from a different user set.

   This way, the output clearly shows which comparisons are accepted or rejected and gives a practical view of how the distance behaves for real vs non-matching signatures.

PYTHON IMPLEMENTATION

```python
# Upload + extract the dataset archive (.rar)
from google.colab import files
up = files.upload()

!apt-get -qq update
!apt-get -qq install -y unrar > /dev/null

import os, glob
rar = list(up.keys())[0]
os.makedirs('data', exist_ok=True)
!unrar x -o+ "$rar" data/ > /dev/null

all_files = sorted([p for p in glob.glob('data/**/*', recursive=True) if os.path.isfile(p)])
print('Extracted files:', len(all_files))
print('Sample files:')
print('\n'.join(all_files[:20]))
```

```
Choose Files U1 And U2.rar
U1 And U2.rar(application/x-compressed) - 130580 bytes, last modified: 12/17/2025 - 100% done
Saving U1 And U2.rar to U1 And U2.rar
W: Skipping acquire of configured file 'main/source/Sources' as repository 'https://r2u.stat.illinois.edu/ubuntu jammy InRelease' does not seem to provide it (sources.list entry misspelt?)
Extracted files: 80
Sample files:
data/U1 And U2/U1S1.TXT
data/U1 And U2/U1S10.TXT
data/U1 And U2/U1S11.TXT
data/U1 And U2/U1S12.TXT
data/U1 And U2/U1S13.TXT
data/U1 And U2/U1S14.TXT
data/U1 And U2/U1S15.TXT
data/U1 And U2/U1S16.TXT
data/U1 And U2/U1S17.TXT
data/U1 And U2/U1S18.TXT
data/U1 And U2/U1S19.TXT
data/U1 And U2/U1S2.TXT
data/U1 And U2/U1S20.TXT
data/U1 And U2/U1S21.TXT
data/U1 And U2/U1S22.TXT
data/U1 And U2/U1S23.TXT
data/U1 And U2/U1S24.TXT
data/U1 And U2/U1S25.TXT
data/U1 And U2/U1S26.TXT
data/U1 And U2/U1S27.TXT
```

```python
# Read signature points (x,y,t,s) from a text file
import re

def load_points(fp):
    pts = []
    with open(fp, 'r', errors='ignore') as f:
        for ln in f:
            parts = re.split(r'[\s,;]+', ln.strip())
            if len(parts) < 4:
                continue
            try:
                x, y, t, s = map(int, parts[:4])
            except:
                continue
            pts.append((x, y, t, s))
    return pts
```

```python
# Split into strokes using pen state (s=1 down, s=0 up)
def strokes_from_points(pts, min_len=3):
    strokes, cur = [], []
    for x, y, t, s in pts:
        if s == 1:
            cur.append((x, y, t, s))
        else:
            if len(cur) >= min_len:
                strokes.append(cur)
            cur = []
    if len(cur) >= min_len:
        strokes.append(cur)
    return strokes
```

```python
# Convert stroke to a string-token list using local extrema (xmin/xmax/ymin/ymax)
def extrema_tokens(stroke):
    xs = [p[0] for p in stroke]
    ys = [p[1] for p in stroke]
    out = []
    for i in range(1, len(stroke)-1):
        if xs[i] < xs[i-1] and xs[i] < xs[i+1]: out.append('xmin')
        if xs[i] > xs[i-1] and xs[i] > xs[i+1]: out.append('xmax')
        if ys[i] < ys[i-1] and ys[i] < ys[i+1]: out.append('ymin')
        if ys[i] > ys[i-1] and ys[i] > ys[i+1]: out.append('ymax')
    return out

def signature_sequence(fp):
    pts = load_points(fp)
    seq = []
    for st in strokes_from_points(pts):
        tok = extrema_tokens(st)
        if tok:
            seq.append(tok)
    return seq
```

```python
# Levenshtein distance for symbol lists
def lev(a, b):
    m, n = len(a), len(b)
    prev = list(range(n+1))
    cur = [0]*(n+1)
    for i in range(1, m+1):
        cur[0] = i
        ai = a[i-1]
        for j in range(1, n+1):
            cost = 0 if ai == b[j-1] else 1
            cur[j] = min(cur[j-1] + 1, prev[j] + 1, prev[j-1] + cost)
        prev, cur = cur, prev
    return prev[n]
```

```python
# Adapted MED between signature stroke-sequences
def med_signature(seqA, seqB):
    m, n = len(seqA), len(seqB)
    D = [[0]*(n+1) for _ in range(m+1)]

    for i in range(1, m+1):
        D[i][0] = D[i-1][0] + len(seqA[i-1])
    for j in range(1, n+1):
        D[0][j] = D[0][j-1] + len(seqB[j-1])

    for i in range(1, m+1):
        a = seqA[i-1]
        la = len(a)
        for j in range(1, n+1):
            b = seqB[j-1]
            lb = len(b)
            D[i][j] = min(
                D[i-1][j] + la,          # delete stroke
                D[i][j-1] + lb,          # insert stroke
                D[i-1][j-1] + lev(a, b)  # replace stroke
            )
    return D[m][n]

def dist_norm(seqA, seqB):
    denom = sum(len(s) for s in seqA) + sum(len(s) for s in seqB)
    return (med_signature(seqA, seqB) / denom) if denom > 0 else 0.0

def score(fp1, fp2):
    return dist_norm(signature_sequence(fp1), signature_sequence(fp2))
```

```python
# Auto-detect sets (U1, U2, ...) and sample indices (S1, S2, ...)
import os

def parse_set_sample(path):
    name = os.path.basename(path)
    m = re.search(r'(?i)(u\d+)\s*[^\d]*s\s*(\d+)', name)
    if m:
        return m.group(1).upper(), int(m.group(2))
    # try folder name like U1
    parts = os.path.normpath(path).split(os.sep)
    for part in parts:
        if re.fullmatch(r'(?i)u\d+', part):
            return part.upper(), None
    return 'UNK', None

sig_files = [p for p in all_files if os.path.splitext(p)[1].lower() in {'.txt','.dat','.csv'}]
by_set = {}
for p in sig_files:
    s, idx = parse_set_sample(p)
    by_set.setdefault(s, []).append((idx, p))

sets = sorted([s for s in by_set if s != 'UNK'], key=lambda x: int(x[1:])) + (['UNK'] if 'UNK' in by_set else [])
print('Detected sets:', sets)
for s in sets:
    print(s, 'files:', len(by_set[s]))
```

```
Detected sets: ['U1', 'U2']
U1 files: 40
U2 files: 40
```

```python
# Evaluate: Genuine vs Genuine and Genuine vs Forged (if available)
THRESHOLD = 0.35

def split_genuine_forged(pairs):
    # pairs: (idx, path); if idx exists and max>=40, use 1..20 genuine, 21..40 forged
    idxs = [i for i,_ in pairs if i is not None]
    if idxs and max(idxs) >= 40:
        genu = [p for i,p in pairs if i is not None and 1 <= i <= 20]
        forg = [p for i,p in pairs if i is not None and 21 <= i <= 40]
        rest = [p for i,p in pairs if i is None or i > 40]
        return genu, forg, rest
    # fallback: all files are 'genuine' (no explicit forgeries)
    return [p for _,p in pairs], [], []

def report_pair(label, a, b, d):
    dec = 'ACCEPT' if d <= THRESHOLD else 'REJECT'
    print(f'{label}: {os.path.basename(a)} vs {os.path.basename(b)} -> {d:.4f}  {dec}')
    return dec

for s in sets:
    pairs = sorted(by_set[s], key=lambda t: (t[0] is None, t[0] if t[0] is not None else 10**9, t[1]))
    genu, forg, extra = split_genuine_forged(pairs)

    print('\n' + '='*55)
    print(f'=== {s} ===')
    print('Total files:', len(pairs), '| genuine:', len(genu), '| forged:', len(forg), '| extra:', len(extra))

    if len(genu) < 2:
        print('(Not enough genuine samples to compare)')
        continue

    # pick template as first genuine
    template = genu[0]
    tseq = signature_sequence(template)
    print('Claimed set:', s)
    print('Template file:', template)
    print('Template tokens (first 20):', (tseq[0][:20] if len(tseq) else []))
    print('Template strokes:', len(tseq))
```

```python
# genuine vs genuine (all pairs inside genuine)
print('\n----- Genuine vs Genuine -----')
gg_accept = gg_reject = 0
for i in range(len(genu)):
    for j in range(i+1, len(genu)):
        d = score(genu[i], genu[j])
        dec = report_pair('GG', genu[i], genu[j], d)
        gg_accept += (dec == 'ACCEPT')
        gg_reject += (dec == 'REJECT')
print('GG summary: ACCEPT=', gg_accept, 'REJECT=', gg_reject)

# genuine vs forged (if forged exists) else impostor from other sets
if len(forg) > 0:
    print('\n----- Genuine vs Forged -----')
    gf_accept = gf_reject = 0
    for g in genu:
        for f in forg:
            d = score(g, f)
            dec = report_pair('GF', g, f, d)
            gf_accept += (dec == 'ACCEPT')
            gf_reject += (dec == 'REJECT')
    print('GF summary: ACCEPT=', gf_accept, 'REJECT=', gf_reject)
else:
    # impostor: use first file from next set (if any)
    other_sets = [x for x in sets if x != s and len(by_set.get(x, [])) > 0]
    if other_sets:
        imp_set = other_sets[0]
        imp_path = sorted(by_set[imp_set])[0][1]
        print(f'\n----- Genuine vs Impostor (from {imp_set}) -----')
        gi_accept = gi_reject = 0
        for g in genu:
            d = score(g, imp_path)
            dec = report_pair('GI', g, imp_path, d)
            gi_accept += (dec == 'ACCEPT')
            gi_reject += (dec == 'REJECT')
        print('GI summary: ACCEPT=', gi_accept, 'REJECT=', gi_reject)
    else:
        print('\n(No forged data and no other set for impostor)')
```

OUTPUT IMAGES

```
=========================================================
=== U1 ===
Total files: 40 | genuine: 20 | forged: 20 | extra: 0
Claimed set: U1
Template file: data/U1 And U2/U1S1.TXT
Template tokens (first 20): ['xmin', 'ymax', 'xmax', 'ymin']
Template strokes: 4
```

```
GG: U1S13.TXT vs U1S18.TXT -> 0.3529  REJECT
GG: U1S13.TXT vs U1S19.TXT -> 0.2927  ACCEPT
GG: U1S13.TXT vs U1S20.TXT -> 0.3333  ACCEPT
GG: U1S14.TXT vs U1S15.TXT -> 0.3125  ACCEPT
GG: U1S14.TXT vs U1S16.TXT -> 0.2414  ACCEPT
GG: U1S14.TXT vs U1S17.TXT -> 0.2143  ACCEPT
GG: U1S14.TXT vs U1S18.TXT -> 0.2069  ACCEPT
GG: U1S14.TXT vs U1S19.TXT -> 0.3333  ACCEPT
GG: U1S14.TXT vs U1S20.TXT -> 0.1786  ACCEPT
GG: U1S15.TXT vs U1S16.TXT -> 0.3333  ACCEPT
GG: U1S15.TXT vs U1S17.TXT -> 0.3125  ACCEPT
GG: U1S15.TXT vs U1S18.TXT -> 0.3333  ACCEPT
GG: U1S15.TXT vs U1S19.TXT -> 0.3000  ACCEPT
GG: U1S15.TXT vs U1S20.TXT -> 0.2812  ACCEPT
GG: U1S16.TXT vs U1S17.TXT -> 0.2414  ACCEPT
GG: U1S16.TXT vs U1S18.TXT -> 0.2667  ACCEPT
GG: U1S16.TXT vs U1S19.TXT -> 0.3243  ACCEPT
GG: U1S16.TXT vs U1S20.TXT -> 0.2414  ACCEPT
GG: U1S17.TXT vs U1S18.TXT -> 0.2759  ACCEPT
GG: U1S17.TXT vs U1S19.TXT -> 0.3056  ACCEPT
GG: U1S17.TXT vs U1S20.TXT -> 0.2857  ACCEPT
GG: U1S18.TXT vs U1S19.TXT -> 0.2973  ACCEPT
GG: U1S18.TXT vs U1S20.TXT -> 0.2759  ACCEPT
GG: U1S19.TXT vs U1S20.TXT -> 0.3056  ACCEPT
GG summary: ACCEPT= 145 REJECT= 45
```

```
----- Genuine vs Forged -----
GF: U1S1.TXT vs U1S21.TXT -> 0.3667  REJECT
GF: U1S1.TXT vs U1S22.TXT -> 0.3182  ACCEPT
GF: U1S1.TXT vs U1S23.TXT -> 0.5238  REJECT
GF: U1S1.TXT vs U1S24.TXT -> 0.3571  REJECT
GF: U1S1.TXT vs U1S25.TXT -> 0.4286  REJECT
```

```
GF: U1S20.TXT vs U1S26.TXT -> 0.3462  ACCEPT
GF: U1S20.TXT vs U1S27.TXT -> 0.3030  ACCEPT
GF: U1S20.TXT vs U1S28.TXT -> 0.4231  REJECT
GF: U1S20.TXT vs U1S29.TXT -> 0.4138  REJECT
GF: U1S20.TXT vs U1S30.TXT -> 0.3750  REJECT
GF: U1S20.TXT vs U1S31.TXT -> 0.6000  REJECT
GF: U1S20.TXT vs U1S32.TXT -> 0.5714  REJECT
GF: U1S20.TXT vs U1S33.TXT -> 0.3333  ACCEPT
GF: U1S20.TXT vs U1S34.TXT -> 0.4412  REJECT
GF: U1S20.TXT vs U1S35.TXT -> 0.2500  ACCEPT
GF: U1S20.TXT vs U1S36.TXT -> 0.3462  ACCEPT
GF: U1S20.TXT vs U1S37.TXT -> 0.4074  REJECT
GF: U1S20.TXT vs U1S38.TXT -> 0.4242  REJECT
GF: U1S20.TXT vs U1S39.TXT -> 0.4286  REJECT
GF: U1S20.TXT vs U1S40.TXT -> 0.4242  REJECT
GF summary: ACCEPT= 127 REJECT= 273
```

```
=========================================================
=== U2 ===
Total files: 40 | genuine: 20 | forged: 20 | extra: 0
Claimed set: U2
Template file: data/U1 And U2/U2S1.TXT
Template tokens (first 20): ['xmin', 'ymin']
Template strokes: 5
```

```
----- Genuine vs Genuine -----
GG: U2S1.TXT vs U2S2.TXT -> 0.3284  ACCEPT
GG: U2S1.TXT vs U2S3.TXT -> 0.3239  ACCEPT
GG: U2S1.TXT vs U2S4.TXT -> 0.3889  REJECT
GG: U2S1.TXT vs U2S5.TXT -> 0.3000  ACCEPT
GG: U2S1.TXT vs U2S6.TXT -> 0.2500  ACCEPT
GG: U2S1.TXT vs U2S7.TXT -> 0.4286  REJECT
GG: U2S1.TXT vs U2S8.TXT -> 0.3529  REJECT
GG: U2S1.TXT vs U2S9.TXT -> 0.3662  REJECT
GG: U2S1.TXT vs U2S10.TXT -> 0.4091  REJECT
```

```
GG: U2S16.TXT vs U2S18.TXT -> 0.2571  ACCEPT
GG: U2S16.TXT vs U2S19.TXT -> 0.1644  ACCEPT
GG: U2S16.TXT vs U2S20.TXT -> 0.1918  ACCEPT
GG: U2S17.TXT vs U2S18.TXT -> 0.2609  ACCEPT
GG: U2S17.TXT vs U2S19.TXT -> 0.2500  ACCEPT
GG: U2S17.TXT vs U2S20.TXT -> 0.1944  ACCEPT
GG: U2S18.TXT vs U2S19.TXT -> 0.2754  ACCEPT
GG: U2S18.TXT vs U2S20.TXT -> 0.2609  ACCEPT
GG: U2S19.TXT vs U2S20.TXT -> 0.2222  ACCEPT
GG summary: ACCEPT= 116 REJECT= 74
```

```
----- Genuine vs Forged -----
GF: U2S1.TXT vs U2S21.TXT -> 0.3913  REJECT
GF: U2S1.TXT vs U2S22.TXT -> 0.4225  REJECT
GF: U2S1.TXT vs U2S23.TXT -> 0.3538  REJECT
GF: U2S1.TXT vs U2S24.TXT -> 0.3824  REJECT
GF: U2S1.TXT vs U2S25.TXT -> 0.3731  REJECT
GF: U2S1.TXT vs U2S26.TXT -> 0.3220  ACCEPT
GF: U2S1.TXT vs U2S27.TXT -> 0.4098  REJECT
GF: U2S1.TXT vs U2S28.TXT -> 0.3692  REJECT
```

```
GF: U2S20.TXT vs U2S32.TXT -> 0.4154  REJECT
GF: U2S20.TXT vs U2S33.TXT -> 0.3676  REJECT
GF: U2S20.TXT vs U2S34.TXT -> 0.3235  ACCEPT
GF: U2S20.TXT vs U2S35.TXT -> 0.3056  ACCEPT
GF: U2S20.TXT vs U2S36.TXT -> 0.3182  ACCEPT
GF: U2S20.TXT vs U2S37.TXT -> 0.4493  REJECT
GF: U2S20.TXT vs U2S38.TXT -> 0.3768  REJECT
GF: U2S20.TXT vs U2S39.TXT -> 0.4143  REJECT
GF: U2S20.TXT vs U2S40.TXT -> 0.4286  REJECT
GF summary: ACCEPT= 115 REJECT= 285
```

Evaluation & Comparison

The experiments were run separately for **U1** and **U2**, each containing **40 samples**: the first **20** treated as genuine and the next **20** treated as forged. A fixed decision threshold of **0.35** was used, meaning that distances below or equal to 0.35 are accepted.

1) U1 results:

Genuine vs Genuine (GG)
For U1, all genuine pairs among the 20 genuine samples were compared (190 pairs total).

- Accepted: 145
- Rejected: 45

So most genuine comparisons were accepted, but there is still a noticeable number of rejections. This is visible in the output: some genuine distances are comfortably low (e.g., around 0.10–0.25), while others jump above the threshold (0.37–0.47). That tells us U1's genuine signatures are not perfectly consistent—some samples differ enough in stroke structure or extrema order that the edit distance becomes larger.

Genuine vs Forged (GF)
Each genuine sample was compared with each forged sample (400 comparisons total).

- Accepted: 127
- Rejected: 273

Most forged comparisons were rejected, which is good. However, there are still many forged comparisons that ended up under the threshold and were accepted (for example, distances around 0.25–0.33 show up quite often). This suggests that some forged samples share similar "extrema patterns" with the genuine ones, at least under this simplified representation.

Overall impression for U1:
U1 shows strong genuine matching in many cases but also has several genuine rejections and a non-trivial number of forged acceptances. In other words, the method recognizes many genuine similarities, but some forgeries still "look" close in the symbolic domain.

2) U2 results:
Genuine vs Genuine (GG)
Again, 190 genuine pairs were compared.

- Accepted: 116
- Rejected: 74

Compared to U1, U2 has fewer accepted genuine pairs and more rejections. This indicates that U2's genuine signatures vary more according to the features we used (strokes + local extrema tokens). You can see it in the distances: there are many genuine scores near the boundary and quite a few above it.

Genuine vs Forged (GF)
400 genuine–forged comparisons were performed.

- Accepted: 115
- Rejected: 285

Here the forged acceptances are slightly lower than U1. So, for U2, the system rejects forged signatures a bit more reliably, but at the cost of rejecting more genuine comparisons as well.

Overall impression for U2:
U2 looks like a "tougher" user for this method: genuine samples are less consistent under the extrema-token representation, but at the same time many forgeries are pushed to larger distances and get rejected.

3) Comparison between U1 and U2
Putting both users side-by-side:

- **Genuine matching:** U1 performs better (more genuine pairs accepted) than U2.
- **Forgery rejection:** U2 performs slightly better (fewer forged comparisons accepted) than U1.

This is a classic trade-off you can observe even without changing the algorithm: one fixed threshold does not behave equally well for every signer. Some people naturally sign very consistently, while others have more variation from sample to sample. That difference shows up directly in the distance values.

Conclusion

This project implemented a dynamic signature verification approach based on the lecture-note idea of converting signatures into symbolic sequences and comparing them using an adapted edit-distance framework. In the experiments, the method produced reasonable separation between genuine and forged comparisons, especially because most forged pairs were rejected in both U1 and U2.

At the same time, the results also show the limitations of using a simple symbolic representation based only on local extrema. Some genuine signatures were rejected because natural variation changed the stroke segmentation or the order of extrema tokens. On the other hand, several forged samples were accepted because the extrema patterns were still similar enough to fall under the threshold.

Overall, the approach works well as a clear, explainable baseline: it is easy to describe, the distance score is intuitive, and the output makes it obvious why a decision is accepted or reject. If this were pushed further, the most practical improvements would be using more than one template per user, adding a small amount of preprocessing (like smoothing), and allowing user-specific thresholds—because the comparison results clearly depend on the signer.

References

- OpenCV Python Documentation: https://docs.opencv.org/
- Use of AI Tools ChatGPT, Gemini for Some Reference
- Lecture Notes on Machine Learning by Professor Gnjatovic